Accessible lecture slides for blind students

COMP 8117 SPLASH proposal and problem definition

The problem

Blind and low vision students read course material with a screen reader such as NVDA, JAWS or VoiceOver. The screen reader turns the file into speech, and into braille on a braille display or a device like the APH Monarch.

This only works if the file says what its content means. Most lecture slides do not. A PDF stores where text and pictures go on the page, but not what they are. Formulas, tables and diagrams are usually saved as pictures with no description. So the screen reader has nothing to say about them.

Today a person fixes these files by hand. That is slow, so blind students often get their material days after everyone else.

Problem statement. Lecture slides lose their meaning for screen reader users because formulas, tables and diagrams are stored as unlabelled pictures. Fixing them by hand is slow. We want to find out how much of that meaning can be added back automatically and accurately, so a blind student can use a lecture on the same day as their classmates.

How a file reaches a blind student

1. The PDF holds drawing instructions. It can also hold hidden tags that say what each part is (heading, table, formula) and in which order to read it. Most PDFs have no tags.
2. The PDF viewer builds a list of items for the screen reader from those tags. Without tags it guesses.
3. The screen reader reads that list as speech. The same text goes to a braille display through Liblouis, and math is read properly through MathCAT if it is stored as MathML.

Steps 2 and 3 already work well. Step 1 is where things break, because the meaning is missing from the file.

What already exists

Devices. The APH and HumanWare Monarch is a braille tablet with about 3,840 pins. It shows braille and simple tactile pictures, and it can open PDF and image files. APH says it works best with simple images. A detailed lecture diagram has to be simplified by a person first, using tools like TactileView. The Dot Pad claims to do this with AI, but we found no independent test of it.



The Monarch braille tablet. The grey area in the middle is the pin surface, with the braille keys below it. Photo from APH.



The Monarch showing a math graph as raised pins. Photo from HumanWare.

Tools that add tags. Adobe Acrobat auto tagging, Equidox, CommonLook and DocAble add tags or convert math automatically. They work on simple documents. Research reports they still break on slides, multi column layouts and formulas, and reading order is often still set by hand.

Research.

- AI remediation of STEM PDFs (ACM ASSETS 2022) adds structure and formulas automatically but still makes tagging mistakes on complex layouts.
- A study of human versus AI alt text for computer science figures found that automatic quality scores miss the errors that matter to screen reader users.

- FigurA11y, AlternAtIve and AltGosling use AI to describe scientific figures. Their tests with blind users are small.
- Olli and Chart Reader from MIT give layered descriptions (overview first, then details) but only for charts.
- VizAbility lets blind users ask questions about a chart.
- Recent work on automatic tactile graphics (ASSETS 2025, TactileEval) is early and lists refreshable displays as future work.

The gap. The pieces exist but separately, and mostly for papers or charts. We found nobody who combines them for lecture slides, checks the accuracy, and tests the result through what a screen reader actually says.

Getting a lecture PDF onto the Monarch

According to the APH Monarch FAQ, the Monarch opens braille files (BRF, BRL, eBraille), books (EPUB, DAISY), documents (PDF, Word, TXT), images (PNG, JPG and PDF pages as pictures) and supports MathML. Different apps on the device open different files.

The device does not turn a normal document into proper braille by itself. That step happens before the file reaches it. So a lecture PDF has three possible paths.

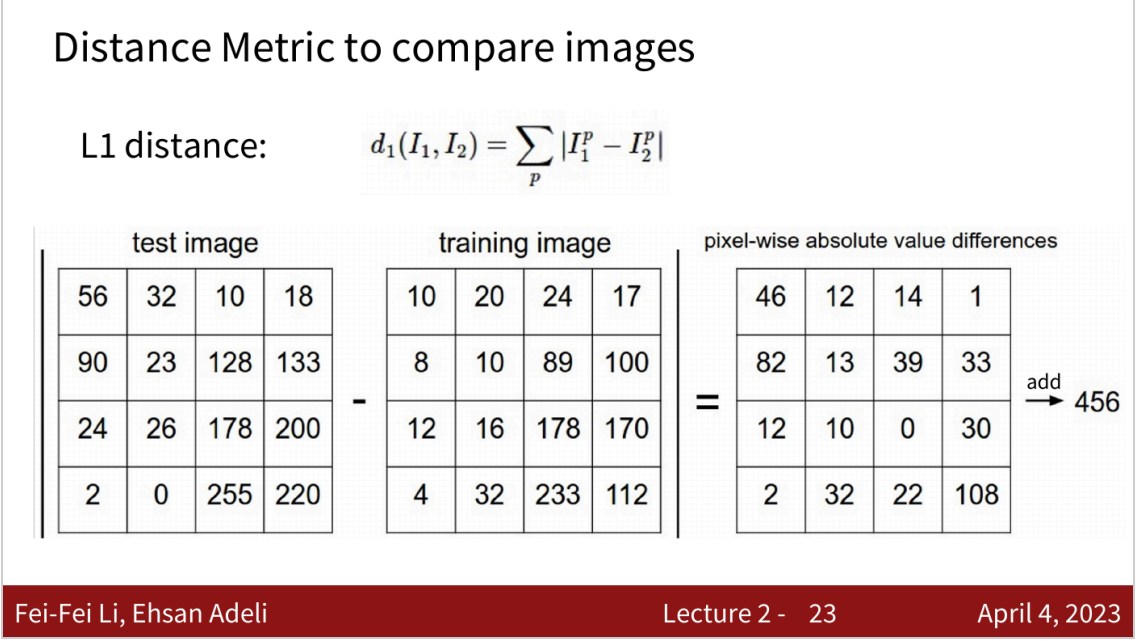
1. **As braille text.** A transcriber turns the PDF into a braille file (BRF) with braille software, and APH's converter can turn that into eBraille. This is good for text, but someone still has to deal with the formulas, tables and diagrams by hand.
2. **As a picture.** The slide is opened as an image in the Tactile Viewer and raised in pins as it is. For slide 23 this would give 48 tiny numbers and a formula squeezed into about 96 by 40 dots, which a finger cannot read. APH says simple images work best.
3. **As a structured file.** A file that already has headings, descriptions, real tables and MathML, like our prototype output, can be read through a screen reader on the Monarch or any braille display, and the braille comes from it automatically.

Path 3 is the one our project feeds. Turning a cleaned up diagram into a simple tactile picture for path 2 is future work.

Feasibility check we ran

We took a real public lecture, Stanford CS231n Lecture 2 (98 slides), and checked what a screen reader can get from it.

The file has no tags and no structure at all. Slide 23 contains two pictures and no descriptions.

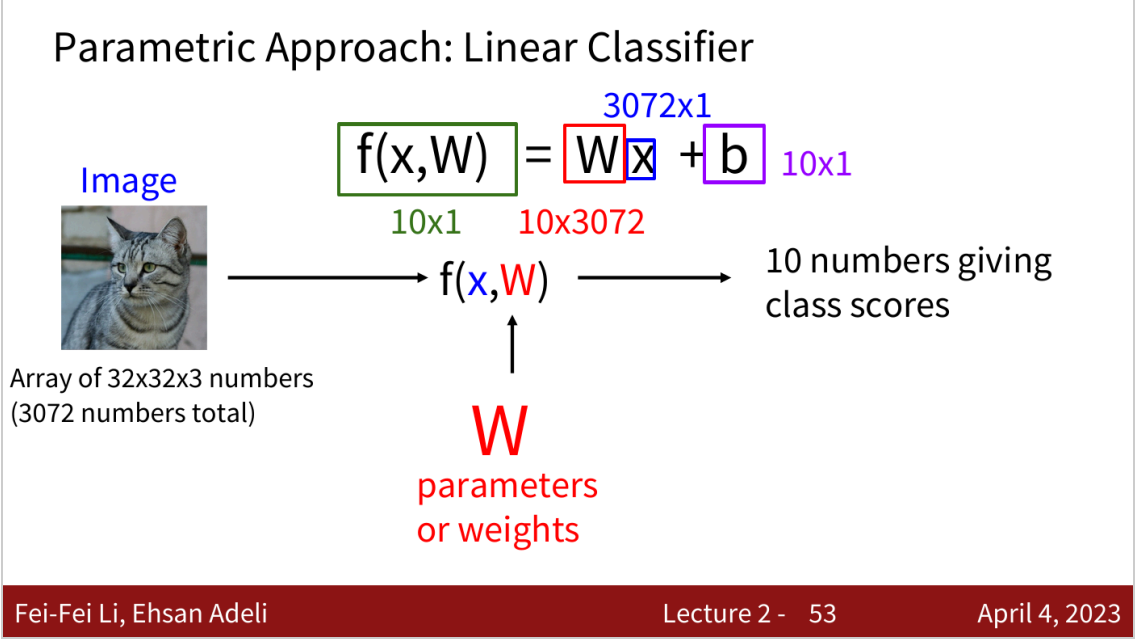


Slide 23 as a sighted student sees it

This is all the text a screen reader can get from slide 23.

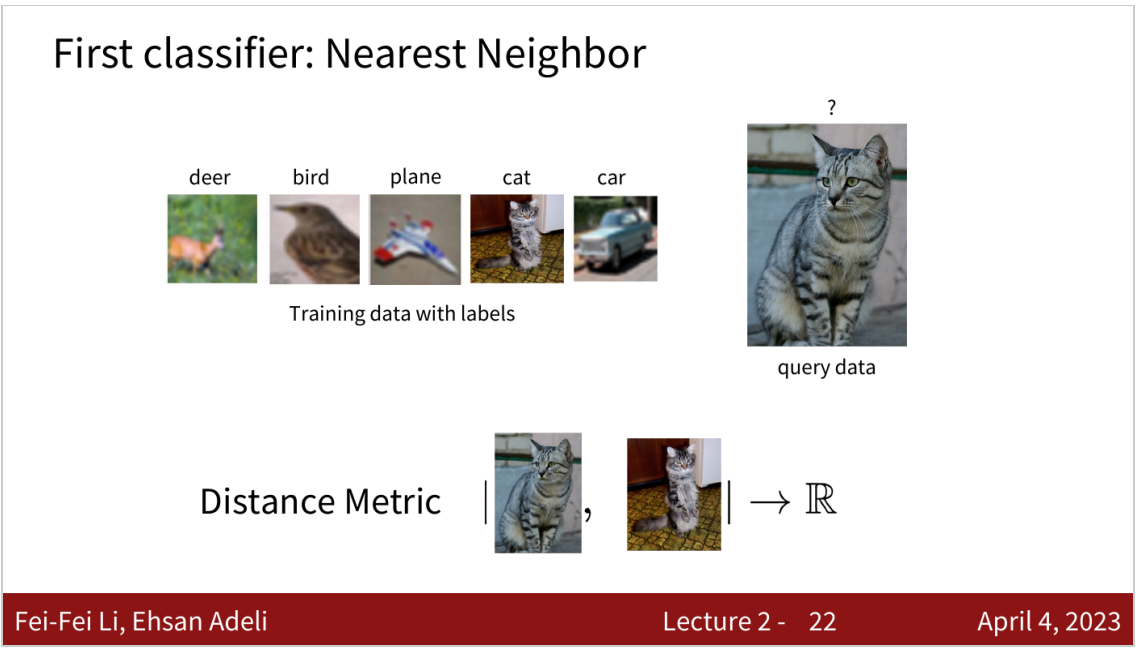
```
Distance Metric to compare images
L1 distance:
add
Fei-Fei Li, Ehsan Adeli
Lecture 2 - 23
April 4, 2023
```

The formula, all 48 numbers and the answer 456 are missing. A blind student hears “L1 distance: add” and nothing else.



Slide 53 as a sighted student sees it

For this slide a screen reader gets scraps in the wrong order, like “f(x,W) = W x + b 10x1, 10x1, 10x3072”. The sizes are not attached to anything and the arrows are lost.



Slide 22 as a sighted student sees it

For this slide it gets “deer bird plane cat car” as a list of words. The six photos are silent and the main idea is lost.

First prototype

We built a small pipeline to add the missing meaning to these three slides.

1. It reads the slide title from the PDF.
2. An AI step describes each slide in layers (overview, parts, how the parts relate), writes formulas as LaTeX and reads the numbers from tables. In this first run Claude did this step by looking at the slide images.
3. The pipeline turns formulas into MathML so screen readers can speak them.
4. It builds an accessible page with real headings, lists, tables and math, and can also export it as a tagged PDF.

The code is in the prototype folder (make_accessible.py and slides.json).

What came out

We checked the result with Chrome's accessibility tree, which is what a screen reader receives.

	Original PDF	**Our version**
Tagged	No	Yes
Headings to jump between	0	17
Formulas a screen reader can speak	0	4
Real tables	0	3 (48 numbers readable cell by cell)
Diagram descriptions	0	3, each with overview, parts and relationships

This is part of what a screen reader now gets for slide 23.

```
heading 2   Slide 23: Distance Metric to compare images
heading 3   Overview
            A worked example of the L1 distance between two images ...
            add up to 456.
heading 3   Parts
            list of 5 items
heading 3   How the parts relate
            ... the top left cell is 56 minus 10, which is 46 ...
heading 3   Formulas
            math   d sub 1 ( I sub 1 , I sub 2 ) = sum over p | I 1 p − I 2 p |
heading 3   Tables
            table  Test image pixel values (4 by 4)
```

One finding surprised us. When we exported the same page as a tagged PDF, headings, lists and tables kept their tags, but the formulas did not get a Formula tag. So a PDF loses the math meaning while HTML keeps it. This supports our choice of HTML as the main output.

What this does not prove yet

- The AI step was done by Claude by hand for three slides. The real pipeline must do it automatically, and we must measure how often it is wrong.
- We read Chrome's accessibility tree, not real NVDA speech. NVDA only runs on Windows. Next we record NVDA or VoiceOver output with Guidepup.
- No blind student has used it yet.
- We have not yet run the same slides through Adobe or Equidox to compare.

Proposal

We will build a pipeline that turns lecture slides into accessible HTML. It will fix structure and reading order, describe diagrams in layers, turn formulas into MathML, rebuild tables, and flag anything it is unsure about for a person to check. We will not build braille output, because existing tools do that once the content is structured. Tactile pictures for the Monarch are future work unless we can borrow a device.

How we will prove it works.

1. Accessibility checks on the original and our version.
2. What NVDA or VoiceOver actually says, recorded with Guidepup.
3. An AI agent that only hears the screen reader output and tries real tasks, like finding a formula or reading a table value.
4. Accuracy of the AI descriptions, checked by people.
5. A few real screen reader users doing the same tasks, to check the agent agrees with them.
6. Time to produce an accessible file compared with fixing it by hand.

Steps

1. Email Student Accessibility Services to confirm how long conversions take and which content is hardest.
2. Collect real lecture files from two or three instructors.
3. Run them through Adobe auto tagging and record what a screen reader says, so we have a fair baseline.
4. Automate the AI step and measure its accuracy.
5. Set up NVDA or VoiceOver with Guidepup and the AI agent test.
6. Run sessions with real screen reader users.
7. Measure before and after, write up the results, and prepare the defense.

Team

- Research and users. Interviews, user sessions, before and after measurement.

- Document processing. Reading PDFs, structure, reading order, tables, HTML.
- AI and math. Descriptions, formulas, accuracy checks.
- Platform and quality. Pipeline, deployment, CI, the screen reader test setup, budget and decisions log.

Use of AI tools

We used Claude to research existing work, build the first prototype and draft this document. The team checked the claims and remains responsible for them.